

Merge Intervals Concept

✓ 1. Insert Interval

Problem: Given a list of non-overlapping intervals sorted by start time, insert a new interval and merge if necessary.

🧠 Approach:

- Traverse through the original intervals.
- Add all intervals that **end before** the new interval starts.
- Merge all overlapping intervals with the new interval.
- Add all intervals that **start after** the new merged interval ends.

✍ Steps:

1. Traverse intervals:
 - `interval.end < newInterval.start` → push to result.
2. While overlapping:
 - Update `newInterval` with `min(start)` and `max(end)`.
3. Add merged `newInterval` to result.
4. Add remaining intervals.

✓ **Time: $O(n)$, Space: $O(n)$**

✓ 2. Erase Overlapping Intervals (Non-overlapping Intervals)

Leetcode 435: Remove the minimum number of intervals so the rest are non-overlapping.

🧠 Approach:

- **Greedy:** Sort by end time.
- Always keep the interval that ends earliest (maximizes room for others).

✍ Steps:

1. Sort intervals by end time.

2. Track `prevEnd = intervals[0][1]`.

3. For each interval:

- If `start >= prevEnd`: no overlap, update `prevEnd`.
- Else: overlap → increment count.

✓ **Time: $O(n \log n)$, Space: $O(1)$**

✓ 3. Line Sweep Concept

A technique used to solve interval and geometry problems by sweeping a line across sorted "events".

🧠 Core Idea:

- Convert intervals into **events**:
 - Start → +1
 - End → -1
- Sort events by time.
- Sweep through, tracking **active intervals** (or overlaps).

🔧 Used In:

- Max overlapping meetings
- Skyline problem
- My Calendar
- CPU load

✓ **Time: $O(n \log n)$ due to sorting**

✓ 4. Interval Intersection

Find intersection between two lists of intervals, both sorted and disjoint internally.

🧠 Approach:

- Use two pointers (`i` and `j`) to scan both lists.

- At each step, compute intersection:
 - `start = max(a.start, b.start)`
 - `end = min(a.end, b.end)`
 - If `start <= end`: intersection exists

Steps:

1. While `i < n` and `j < m`:
 - Check overlap using max/min
 - Advance the pointer with smaller end

✓ Time: $O(n + m)$, Space: $O(k)$

✓ 5. Merge Intervals

Merge all overlapping intervals in a list.

Approach:

1. Sort intervals by **start time**
2. Initialize result with the first interval
3. Traverse and:
 - If overlap (`current.start <= last.end`) → merge
 - Else → push to result

✓ Time: $O(n \log n)$, Space: $O(n)$

Code Samples

✓ 1. Merge Intervals

● Brute Force:

- Check all pairs for overlap and merge iteratively (inefficient).

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    int n = intervals.size();
    vector<bool> merged(n, false);
    vector<vector<int>> result;

    for (int i = 0; i < n; ++i) {
        if (merged[i]) continue;
        int start = intervals[i][0], end = intervals[i][1];

        for (int j = i + 1; j < n; ++j) {
            if (!merged[j] && !(intervals[j][0] > end || intervals[j][1] <
start)) {
                start = min(start, intervals[j][0]);
                end = max(end, intervals[j][1]);
                merged[j] = true;
            }
        }

        result.push_back({start, end});
    }

    return result;
}
```

✓ Optimal:

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> result;

    for (auto& interval : intervals) {
        if (result.empty() || result.back()[1] < interval[0])
            result.push_back(interval);
        else
            result.back()[1] = max(result.back()[1], interval[1]);
    }

    return result;
}
```

✓ 2. Insert Interval

● Brute Force:

- Add and re-merge all intervals.

```
vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>&
newInterval) {
    intervals.push_back(newInterval);
    return merge(intervals); // reuse merge function from above
}
```

✓ Optimal:

```
vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>&
newInterval) {
    vector<vector<int>> result;
    int i = 0, n = intervals.size();

    // add all intervals before newInterval
    while (i < n && intervals[i][1] < newInterval[0]) {
        result.push_back(intervals[i++]);
    }

    // merge overlapping intervals
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = min(newInterval[0], intervals[i][0]);
        newInterval[1] = max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.push_back(newInterval);

    // add remaining intervals
    while (i < n) result.push_back(intervals[i++]);

    return result;
}
```

✓ 3. Erase Overlapping Intervals

● Brute Force:

- Compare every pair to check overlap, and count.

```
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    int n = intervals.size(), count = 0;
    sort(intervals.begin(), intervals.end());
    vector<bool> removed(n, false);

    for (int i = 0; i < n; ++i) {
        if (removed[i]) continue;
        for (int j = i + 1; j < n; ++j) {
            if (intervals[j][0] < intervals[i][1]) {
                removed[j] = true;
                count++;
            } else {
                break;
            }
        }
    }

    return count;
}
```

✓ Optimal (Greedy):

```
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b){
        return a[1] < b[1]; // sort by end time
    });

    int count = 0, prevEnd = intervals[0][1];
    for (int i = 1; i < intervals.size(); ++i) {
```

```

        if (intervals[i][0] < prevEnd) {
            count++;
        } else {
            prevEnd = intervals[i][1];
        }
    }
    return count;
}

```

✓ 4. Interval Intersection

● Brute Force:

- Check all pairs from both lists.

```

vector<vector<int>> intervalIntersection(vector<vector<int>>& A,
vector<vector<int>>& B) {
    vector<vector<int>> result;

    for (auto& a : A) {
        for (auto& b : B) {
            int start = max(a[0], b[0]);
            int end = min(a[1], b[1]);
            if (start <= end)
                result.push_back({start, end});
        }
    }

    return result;
}

```

✓ Optimal (Two Pointers):

```

vector<vector<int>> intervalIntersection(vector<vector<int>>& A,
vector<vector<int>>& B) {
    vector<vector<int>> result;
    int i = 0, j = 0;

    while (i < A.size() && j < B.size()) {
        int start = max(A[i][0], B[j][0]);
        int end = min(A[i][1], B[j][1]);

        if (start <= end)
            result.push_back({start, end});

        if (A[i][1] < B[j][1])
            i++;
        else
            j++;
    }

    return result;
}

```

How to Recognize Interval Problems

Scenario	Clues / Trigger Words	Likely Problem Type
Scheduling meetings	“no overlaps,” “fit in room,” “find available time”	 Merge Intervals / Meeting Rooms
Calendar booking	“add a new event,” “merge,” “conflict”	 Insert Interval
Removing conflicts	“minimize removals,” “overlapping events”	 Erase Overlapping Intervals
Intersection of busy times	“common free time,” “shared availability”	 Interval Intersection
Find maximum usage	“max concurrent,” “peak load,” “server load”	 Line Sweep
Gaps between events	“find idle time,” “longest free slot”	 Interval Gap, Free Time
Skyline / sweep geometry	“buildings,” “skyline outline”	 Line Sweep (Advanced)

Problem Type Breakdowns with Clues

1. Merge Intervals

- **Clues:**
 - “Merge overlapping”
 - “Combine intervals”
 - “Consolidate time ranges”
- **Example:** $[[1,3], [2,6], [8,10]] \rightarrow [[1,6], [8,10]]$

2. Insert Interval

- **Clues:**
 - “Insert event/meeting”
 - “Update calendar”
 - “Add an interval and merge if needed”
- **Example:**
 - Insert $[4,8]$ into $[[1,3],[6,9]] \rightarrow [[1,3],[4,9]]$

3. Erase Overlapping Intervals

- **Clues:**
 - “Minimum removals”
 - “Make intervals non-overlapping”
 - “Delete conflicts”
 - **Strategy:**
 - Greedy by earliest end time to maximize room for future intervals
-

4. Interval Intersection

- **Clues:**
 - “Find common time slots”
 - “Shared availability”
 - “Intersection of intervals”
 - **Example:**
 - A: [1,3], [5,6]; B: [2,5], [5,7] $\rightarrow$ [2,3], [5,5], [5,6]
-

5. Line Sweep

- **Clues:**
 - “Maximum simultaneous events”
 - “Count active intervals”
 - “When is usage/load highest”
- **Examples:**
 - My Calendar (overlap count)
 - CPU Load (peak usage)



Tips to Tackle Any Interval Problem

 Step	 Description
1. Understand format	Most intervals are <code>[start, end]</code> and either sorted or need sorting.
2. Sort if needed	Especially by start time or end time (greedy strategies).
3. Use Pointers/Loop	For merging, intersection, or comparisons.
4. Apply Greedy	Many problems (like Erase Overlap) are solved using greedy approaches.
5. Be Careful with Boundaries	Always check <code><=</code> or <code><</code> and off-by-one errors.



Practice Prompts (What to Ask Yourself)

- Do intervals **overlap** or are they **disjoint**?
- Is the goal to **merge**, **insert**, **intersect**, or **remove**?
- Are intervals **sorted** or do I need to sort first?
- Am I comparing one list with another (intersection) or just one list?